NevUp AI Trading Coach

Remote Hiring Hackathon 2026 - Kick-Off Packet

Remote · Async	72-Hour Duration	3 Independent Tracks

1. Overview & Global Rules

Welcome to the NevUp Hiring Hackathon 2026. You are building components of NevUp's real product, a behavioral intelligence platform for retail day traders. The three tracks are independent but share a single data contract and authentication scheme. Read this entire document before writing a single line of code.

Format

Property	Detail
Format	Remote, fully async - work from wherever you are
Duration	72 hours from kick-off email timestamp
Tracks	3 independent tracks - pick exactly one
Submission	Live deployment URL + public GitHub repo
Late policy	Disqualified. No exceptions.

Global Rules - Applies to Every Track

- No mock or hardcoded data in any demo. All data must be seeded or live-generated using the provided seed files attached to this email.
- All services must be containerised with Docker + docker-compose. Reviewers must be able to run your full stack with a single docker compose up - no manual steps, no missing env vars, no README footnotes that say 'first install X'.
- Error handling is a first-class requirement. Every external call must have a timeout, retry, or fallback. Unhandled promise rejections and HTTP 500 responses with an empty body will penalise your score.
- Every submission must include a DECISIONS.md file at the repo root - one paragraph per significant architectural decision explaining the why, not the what.
- All three tracks share an identical judging rubric (see below).

Judging Rubric

Criterion	Weight	What Reviewers Look For
Technical Depth	35%	Performance under load, correct algorithmic choices, system design quality.
Product Thinking	30%	Does the solution solve the actual NevUp product problem, not just the literal spec?
Code Quality	20%	Readability, test coverage, separation of concerns, no obvious anti-patterns.
Documentation	15%	DECISIONS.md quality, README clarity, inline comments where non-obvious.

2. Attached Files - What Each Track Needs

Three files are attached to this drive. The table below maps each file to the tracks that require it.

File	Track 1	Track 2	Track 3	Purpose
`nevup_seed_dataset.csv`	**Required**	-	**Required**	388 trades across 10 synthetic traders. Track 1 must load it into the DB on startup. Track 3 must use it as the data source for the mock API.
`nevup_seed_dataset.json`	-	**Required**	-	Same 388 trades structured as nested trader profiles with ground-truth pathology labels. Track 2 uses this for behavioral profiling and the evaluation harness.
`nevup_openapi.yaml`	**Required**	**Reference**	**Required**	OpenAPI 3.0 spec. Track 1 must implement it. Track 3 must consume it. Track 2 uses it as an API reference for the memory contract endpoints.

Seed Dataset Summary

10 synthetic trader profiles | 52 sessions | 388 trades | Date range: Jan – Feb 2026

Each trader has exactly one labelled behavioural pathology - except Avery Chen, who is a control profile with no pathology.

Pathologies present: revenge_trading, overtrading, fomo_entries, plan_non_adherence, premature_exit, loss_running, session_tilt, time_of_day_bias, position_sizing_inconsistency

Seed Trader Reference

Name	userId	Ground-Truth Pathology	Sess.	Trades
Alex Mercer	`f412f236-4edc-47a2-8f54-8763a6ed2ce8`	`revenge_trading`	5	25
Jordan Lee	`fcd434aa-2201-4060-aeb2-f44c77aa0683`	`overtrading`	5	80
Sam Rivera	`84a6a3dd-f2d0-4167-960b-7319a6033d49`	`fomo_entries`	5	30
Casey Kim	`4f2f0816-f350-4684-b6c3-29bbddbb1869`	`plan_non_adherence`	5	35
Morgan Bell	`75076413-e8e8-44ac-861f-c7acb3902d6d`	`premature_exit`	5	35
Taylor Grant	`8effb0f2-f16b-4b5f-87ab-7ffca376f309`	`loss_running`	5	30
Riley Stone	`50dd1053-73b0-43c5-8d0f-d2af88c01451`	`session_tilt`	5	40
Drew Patel	`af2cfc5e-c132-4989-9c12-2913f89271fb`	`time_of_day_bias`	6	48
Quinn Torres	`9419073a-3d58-4ee6-a917-be2d40aecef2`	`position_sizing_inconsistency`	5	35
Avery Chen	`e84ea28c-e5a7-49ef-ac26-a873e32667bd`	- (control, no pathology)	6	30

Before You Start - One-Time Setup

Complete these two steps before writing any code. Both are required regardless of which track you are on.

Step 1 - Seed your database or load your data file

Track 1 (System of Record): load nevup_seed_dataset.csv into your database on container startup via a migration or seed script.
Track 2 (System of AI Engine): load nevup_seed_dataset.json directly - your behavioral profiling and eval harness must read from this file.
Track 3 (System of Engagement): nevup_seed_dataset.csv is the backing data for the mock API server (Step 2 below).

Step 2 - Spin up the mock API server (Track 3 required, others optional for local testing)

No account or installation needed beyond Node.js. Run this single command from the directory containing nevup_openapi.yaml:

```
npx @stoplight/prism-cli mock nevup_openapi.yaml --port 4010
```

All API endpoints are then live at http://localhost:4010. Track 3 must consume this mock server - do not invent your own contract. Track 1 and Track 2 participants may also use it to test their implementations against the shared OpenAPI contract.

3. Shared Data Contract - Canonical Trade Schema

All three tracks implement or consume this exact schema. Any deviation breaks interoperability scoring. Field names, types, and enum values are not negotiable.

```
{
  tradeId:         uuid v4,
  userId:          uuid v4,
  asset:           string,                        // e.g. 'AAPL', 'BTC/USD'
  assetClass:      equity | crypto | forex,
  direction:       long | short,
  entryPrice:      decimal(18,8),
  exitPrice:       decimal(18,8) | null,
  quantity:        decimal(18,8),
  entryAt:         ISO-8601 UTC,
  exitAt:          ISO-8601 UTC | null,
  status:          open | closed | cancelled,
  planAdherence:   integer 1-5 | null,           // user self-rating at close
  emotionalState:  calm | anxious | greedy | fearful | neutral | null,
  entryRationale:  string <= 500 chars | null,
  sessionId:       uuid v4                       // groups trades in one session
}
```

TRACK 1 System of Record Backend Engineering

Problem Statement

Build a real-time trade journal engine with behavioral analytics - defined contracts, proven throughput. NevUp's core stores every trade with its full psychological context: not just P&L, but the emotional state, entry rationale, and plan adherence for each decision. You will build the write path, the async analytics pipeline, and the read API from scratch. The system must be provably correct under load, multi-tenant by default, and expose a contract that the System of AI Engine and System of Engagement tracks can depend on without negotiation.

Seed Data

File: `nevup_seed_dataset.csv` - 388 trades, 10 traders, 52 sessions. Load this into your database on container startup via a migration or seed script. The GET /users/:id/metrics endpoint must return queryable results against this dataset from the moment reviewers run `docker compose up`.

Behavioral Metrics - Compute All Five

These five metrics must be computed asynchronously outside the write path for every closed trade. Their definitions are deterministic - two independent implementations must produce identical outputs from the same input.

#	Metric	Definition
1	Plan Adherence Score	Rolling 10-trade average of planAdherence ratings per user. Queryable via the read API.
2	Revenge Trade Flag	If a trade opens within 90 seconds of a losing close AND emotionalState is anxious or fearful, tag that trade revengeFlag = true.
3	Session Tilt Index	Ratio of (loss-following trades / total trades) in the current sessionId. Updated on each close event.
4	Win Rate by Emotional State	For each emotionalState value, maintain a per-user running win/loss count. Must be queryable via the read API with date range filtering.
5	Overtrading Detector	If a user opens more than 10 trades in any 30-minute sliding window, emit an overtrading event onto the event bus. Must not block the write path.

Hard Requirements

#	Requirement	Detail
1	Idempotent Write API	POST /trades must be idempotent on tradeId. Duplicate submissions must return HTTP 200 with the existing record - not 500 or 409. Prove this in automated tests.

2	Throughput	Sustain 200 concurrent trade-close events/sec for 60 seconds with p95 write latency <= 150ms. Include the k6 or Locust script and the HTML results report in the repo. Justify the 200 figure in DECISIONS.md.
3	Async Pipeline	Behavioral metrics must be computed outside the write path via a message queue (Kafka, RabbitMQ, Redis Streams, or equivalent). HTTP polling is not acceptable.
4	Read API Performance	GET /users/:id/metrics?from=&to=&granularity=hourly|daily|rolling30d must return in <= 200ms p95 against the seeded dataset. Include the query explain plan in DECISIONS.md.
5	Multi-Tenancy Auth	JWT-based. Every endpoint enforces row-level tenancy. Include an automated test that proves cross-tenant reads return HTTP 403.
6	Observability	Structured JSON logs on every request (fields: traceId, userId, latency, statusCode). One health endpoint GET /health that returns queue lag and DB connection state.

Constraints

Any backend language	OpenAPI 3.0 spec required	DB migrations in repo	No ORM hiding N+1s	Load test script required

Submission Checklist

- ☐ ✓ Live deployment URL (service must be running at submission time)
- ☐ ✓ Public GitHub repository
- ☐ ✓ OpenAPI 3.0 specification file
- ☐ ✓ Load test script (k6 or Locust) + HTML results report
- ☐ ✓ DECISIONS.md with architectural rationale
- ☐ ✓ docker-compose.yml - single docker compose up, no manual steps

TRACK 2 System of AI Engine AI / ML Engineering

Problem Statement

Build a stateful trading psychology coach that reasons across sessions - with a verifiable memory layer and a reproducible evaluation harness. The hard part is not calling an LLM. It is building a memory system that is accurate (no hallucinated references to trades that never happened), queryable (the system must explain why it said what it said), and persistent across server restarts. You are building infrastructure, not a chatbot.

Seed Data

File: `nevup_seed_dataset.json` - 10 synthetic trader profiles structured as nested JSON with ground-truth pathology labels. You must use this file as-is. The evaluation harness must be reproducible from this exact dataset. Do not modify or regenerate it.

Memory Contract - Expose All Three Operations

Reviewers will call these endpoints directly during evaluation. They must match this contract exactly.

```
// Persist a session summary after a session ends
PUT /memory/{userId}/sessions/{sessionId}
body: { summary: string, metrics: BehavioralMetrics, tags: string[] }

// Query memory for context before generating a coaching message
GET /memory/{userId}/context?relevantTo={signal}
response: { sessions: SessionSummary[], patternIds: string[] }

// Retrieve a specific session for hallucination audit
GET /memory/{userId}/sessions/{sessionId}
response: raw session record (must exactly match what was stored)
```

Hard Requirements

#	Requirement	Detail
1	Behavioral Profiling	Ingest a trader's history from the seed dataset and produce a structured profile. Each claim (weakness pattern, failure mode, peak window) must cite the specific sessionId and tradeId from which it is derived. Generic statements with no evidence are a grading failure.
2	Session-Level Interventions	Given a stream of sequential trades via POST /session/events, detect any of the 5 behavioral signals from Track 1 and generate a contextual coaching message. Latency: <= 3 seconds p99 on warm, repeated calls. Cold-start timing does not count.
3	Anti-Hallucination Audit	Coaching responses that reference prior sessions must cite real sessionIds. Implement POST /audit: accepts a coaching response body and returns each referenced sessionId with a found | not-found flag. Reviewers will call this endpoint.

4	Memory Persistence	The memory store must survive a docker compose restart without data loss. In-memory stores (plain Python dicts, non-persisted Redis) that reset on restart fail this requirement.
5	Evaluation Harness	Using the 10 provided trader profiles, run your coaching system and produce a classification report showing precision, recall, and F1 per pathology class. Include the script - reviewers will run it from scratch.
6	Streaming Responses	Coaching messages must stream token-by-token via SSE or WebSocket. A single blob after 3 seconds is worse UX than a stream that starts within 400ms.

Constraints

LLM usage allowed	Must use provided seed dataset	Memory must survive restart	Eval script reproducible	Hallucination audit endpoint required

Submission Checklist

- ☐ ✓ Live API endpoint (service must be running at submission time)
- ☐ ✓ Public GitHub repository
- ☐ ✓ Evaluation script + classification report (JSON or HTML)
- ☐ ✓ DECISIONS.md with architectural rationale
- ☐ ✓ Hallucination audit endpoint demonstrated via curl commands in README
- ☐ ✓ docker-compose.yml - single docker compose up, no manual steps

TRACK 3 System of Engagement Frontend Engineering

Problem Statement

Build the post-session debrief and behavioural dashboard - real API integration, measurable performance, no design scaffolding. NevUp's interface must make a trader feel understood, not just informed. You are building two things: a post-session debrief flow and a longitudinal behavioural dashboard. Both must consume live data from the provided mock API. You will be judged equally on interaction design, data storytelling, and engineering quality. A beautiful app that is slow or broken fails. A fast app that is visually generic fails.

Seed Data & Mock API

File: `nevup_seed_dataset.csv` - provides the data your mock API will serve. Spin up the mock server locally using Prism:

```
npx @stoplight/prism-cli mock nevup_openapi.yaml --port 4010
```

All endpoints will be live at http://localhost:4010 once Prism is running. Do not invent your own contract - consume exactly what the spec provides. The mock API exposes:

```
GET  /api/sessions/:id          -> SessionSummary + trade list
GET  /api/users/:id/metrics     -> BehavioralMetrics (timeseries)
GET  /api/users/:id/profile     -> BehavioralProfile
POST /api/sessions/:id/debrief  -> submit debrief answers
SSE  /api/sessions/:id/coaching -> streaming coaching messages (token-by-token)
```

Hard Requirements

#	Requirement	Detail
1	Post-Session Debrief Flow	A 5-step structured review: (1) trade replay list, (2) emotional state tagging per trade, (3) plan adherence rating 1–5, (4) AI coaching message rendered as a live SSE stream - not a static string, (5) key takeaway save via POST /debrief. Each step must have a distinct transition animation. The flow must be completable on a 375px mobile viewport.
2	Behavioural Heatmap	A custom-built canvas or SVG heatmap showing daily trade quality score across a rolling 90-day window. No off-the-shelf calendar heatmap library may be used for this component. Hover must show a session summary tooltip inline. Click must navigate to that session's debrief.
3	Real-Time Coaching Panel	Consume the SSE /coaching endpoint and render tokens as they arrive. The panel must degrade gracefully on connection drop: show a Reconnecting... state with exponential backoff - not a blank or frozen screen.

4	Lighthouse CI	Production build must score >= 90 on Performance, Accessibility, and Best Practices. Include the Lighthouse CI JSON report in the repo. The score must be reproducible by the reviewer running lhci autorun.
5	Error, Loading & Empty States	Every data-fetching component must have an explicit loading skeleton, an error state with a retry action, and an empty state with a helpful prompt. Include screenshots of all three states per component in the README.
6	Keyboard Navigability	The debrief flow must be fully completable without a mouse. Tab order must be logical and visible. Focus management must move to the next step on completion. This will be tested manually by the reviewer.

Constraints

React or Next.js only	No component library	Charting libs OK (except heatmap)	Lighthouse CI required	375px mobile responsive

Submission Checklist

- ☐ ✓ Live deployment URL (service must be running at submission time)
- ☐ ✓ Public GitHub repository
- ☐ ✓ Lighthouse CI JSON report (reproducible via lhci autorun)
- ☐ ✓ README with screenshots of error, loading, and empty states per component
- ☐ ✓ DECISIONS.md with design and architectural rationale
- ☐ ✓ docker-compose.yml - single docker compose up, no manual steps

4. Authentication - JWT Format Specification

All three tracks share a single JWT-based authentication scheme. Every track must implement and validate tokens identically using the signing secret below.

HS256 Signing Secret - paste this exact string into all three tracks
97791d4db2aa5f689c3cc39356ce35762f0a73aa70923039d8ef72a2840a1b02

Token Lifetime

Property	Value
Algorithm	HS256 (HMAC-SHA256)
Expiry	24 hours from iat
Renewal	Re-issue on login - no refresh tokens required for hackathon scope
Clock skew	0 seconds tolerance - use UTC strictly

JWT Header & Payload

```
// Header
{ "alg": "HS256", "typ": "JWT" }

// Payload
{
  "sub":   "f412f236-4edc-47a2-8f54-8763a6ed2ce8",  // userId from seed data
  "iat":   1736150400,
  "exp":   1736237800,
  "role":  "trader",
  "name":  "Alex Mercer"                             // optional
}
```

Claims Reference

Claim	Type / Required	Description
sub	string (UUIDv4) - Required	The authenticated user's userId. Must match userId in all data operations. This is the tenancy enforcement key.
iat	integer (Unix timestamp) - Required	Issued-at time (UTC).
exp	integer (Unix timestamp) - Required	Expiry time (UTC). Tokens expire after 24 hours.

role	string - Required	Always 'trader' for hackathon participants. Reserved for future roles.
name	string - Optional	Human-readable display name. Not used for auth decisions.

Row-Level Tenancy Rule

The single most important auth rule - applies to all three tracks
The sub claim (userId) in the JWT must exactly match the userId in the data being accessed.
Any mismatch must return HTTP 403 - never 404.
Returning 404 or 200 instead of 403 is an automatic point deduction.

```
if (jwt.sub !== requestedUserId) {
  return HTTP 403 { error: 'FORBIDDEN', message: 'Cross-tenant access denied.' }
}
```

Reviewers will run an automated test: (1) issue a JWT for User A, (2) request User B's data using User A's token, (3) expect HTTP 403.

Generating a Test Token

```
const crypto = require('crypto');
const SECRET = '97791d4db2aa5f689c3cc39356ce35762f0a73aa70923039d8ef72a2840a1b02';

function base64url(str) {
  return Buffer.from(str).toString('base64')
    .replace(/=/g, '').replace(/\+/g, '-').replace(/\//g, '_');
}
function signJWT(payload) {
  const header  = base64url(JSON.stringify({ alg: 'HS256', typ: 'JWT' }));
  const body    = base64url(JSON.stringify(payload));
  const sig     = crypto.createHmac('sha256', SECRET)
                        .update(`${header}.${body}`).digest('base64')
                        .replace(/=/g, '').replace(/\+/g, '-').replace(/\//g, '_');
  return `${header}.${body}.${sig}`;
}
const now = Math.floor(Date.now() / 1000);
console.log(signJWT({ sub: 'f412f236-4edc-47a2-8f54-8763a6ed2ce8',
                      iat: now, exp: now + 86400, role: 'trader', name: 'Alex
Mercer' }));
```

Or use jwt.io (browser-based, free) with the same secret to inspect and generate tokens manually.

Validation Checklist - All Tracks

- Verify signature using HS256 + shared secret
- Reject tokens where exp is in the past → HTTP 401
- Reject requests with no Authorization header → HTTP 401

- Reject malformed tokens (bad base64, missing claims) → HTTP 401
- Enforce sub === userId for every data endpoint → HTTP 403 on mismatch
- Include traceId in all 401/403 error responses

Structured Log Fields - All Tracks

```
{  "traceId": "uuid-per-request",  "userId": "from jwt.sub",
   "latency": 142,  "statusCode": 200  }
```

The traceId from the log must match the traceId returned in error response bodies.

5. Submission Instructions

1. Ensure your live deployment URL is accessible at submission time. Services that are down during judging will be scored as non-functional.
2. Submit via the form linked in this email. Include: deployment URL, GitHub repo URL, and your track selection.
3. Late submissions are disqualified - no exceptions, no extensions.
4. Your GitHub repository must be public at submission time. Private repos will not be reviewed.
5. docker compose up must start your full stack with no manual steps. Any submission requiring manual intervention before it runs will be penalised under Code Quality.